

```
import java.util.Objects;
```

```
import java.util.HashSet;
```

```
import java.util.Set;
```

```
public class Employee {
```

```
    // Private variables
```

```
    private int id;
```

```
    private String name;
```

```
    private String department;
```

```
    private double salary;
```

```
    // Constructors
```

```
    public Employee() {
```

```
    }
```

```
    public Employee(int id, String name, String department, double salary) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.department = department;
```

```
        this.salary = salary;
```

```
    }
```

```
// Public getters and setters
```

```
public int getId() {
```

```
    return id;
```

```
}
```

```
public void setId(int id) {
```

```
    this.id = id;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public void setName(String name) {
```

```
    this.name = name;
```

```
}
```

```
public String getDepartment() {
```

```
    return department;
```

```
}
```

```
public void setDepartment(String department) {  
  
    this.department = department;  
  
}
```

```
public double getSalary() {  
  
    return salary;  
  
}
```

```
public void setSalary(double salary) {  
  
    this.salary = salary;  
  
}
```

```
// Override equals method
```

```
@Override
```

```
public boolean equals(Object obj) {  
  
    if (this == obj) {  
  
        return true;  
  
    }  
  
    if (obj == null || getClass() != obj.getClass()) {  
  
        return false;  
  
    }  
  
}
```

```
Employee employee = (Employee) obj;
```

```
        return id == employee.id &&  
  
            Double.compare(employee.salary, salary) == 0 &&  
  
            Objects.equals(name, employee.name) &&  
  
            Objects.equals(department, employee.department);  
  
    }
```

```
// Override hashCode method
```

```
@Override
```

```
public int hashCode() {  
  
    return Objects.hash(id, name, department, salary);  
  
}
```

```
// Override toString method
```

```
@Override
```

```
public String toString() {  
  
    return "Employee{" +  
  
        "id=" + id +  
  
        ", name=" + name + "\" +  
  
        ", department=" + department + "\" +  
  
        ", salary=" + salary +  
  
        '}';  
  
}
```

```
// Main method for testing

public static void main(String[] args) {

    // Create Employee objects using constructor

    Employee emp1 = new Employee(101, "John Doe", "IT", 75000.0);

    Employee emp2 = new Employee(102, "Jane Smith", "HR", 65000.0);

    Employee emp3 = new Employee(103, "Bob Johnson", "Finance", 80000.0);


    // Create object using default constructor and setters

    Employee emp4 = new Employee();

    emp4.setId(104);

    emp4.setName("Alice Brown");

    emp4.setDepartment("Marketing");

    emp4.setSalary(70000.0);


    // Test getters

    System.out.println("=== Testing Getters ===");

    System.out.println("Employee 1 ID: " + emp1.getId());

    System.out.println("Employee 1 Name: " + emp1.getName());

    System.out.println("Employee 1 Department: " + emp1.getDepartment());

    System.out.println("Employee 1 Salary: $" + emp1.getSalary());

    System.out.println();
```

```
// Test setters
```

```
System.out.println("=== Testing Setters ===");
```

```
System.out.println("Before update: " + emp1);
```

```
emp1.setSalary(78000.0);
```

```
emp1.setDepartment("IT - Senior");
```

```
System.out.println("After update: " + emp1);
```

```
System.out.println();
```

```
// Test toString method
```

```
System.out.println("=== Testing toString() ===");
```

```
System.out.println("Employee 1: " + emp1.toString());
```

```
System.out.println("Employee 2: " + emp2);
```

```
System.out.println("Employee 3: " + emp3);
```

```
System.out.println("Employee 4: " + emp4);
```

```
System.out.println();
```

```
// Test equals method
```

```
System.out.println("=== Testing equals() ===");
```

```
Employee emp5 = new Employee(101, "John Doe", "IT - Senior", 78000.0);
```

```
System.out.println("emp1 equals emp2? " + emp1.equals(emp2));
```

```
System.out.println("emp1 equals emp5? " + emp1.equals(emp5));
```

```
System.out.println("emp1 equals null? " + emp1.equals(null));
```

```
System.out.println("emp1 equals itself? " + emp1.equals(emp1));
```

```
System.out.println();
```

```
// Test hashCode method
```

```
System.out.println("=== Testing hashCode() ===");
```

```
System.out.println("emp1 hash code: " + emp1.hashCode());
```

```
System.out.println("emp2 hash code: " + emp2.hashCode());
```

```
System.out.println("emp5 hash code: " + emp5.hashCode());
```

```
System.out.println("emp1 and emp5 have same hash code? " +
```

```
    (emp1.hashCode() == emp5.hashCode()));
```

```
System.out.println();
```

```
// Demonstrate HashSet behavior with equals and hashCode
```

```
System.out.println("=== HashSet Behavior (using equals/hashCode) ===");
```

```
Set<Employee> employeeSet = new HashSet<>();
```

```
employeeSet.add(emp1);
```

```
employeeSet.add(emp2);
```

```
employeeSet.add(emp3);
```

```
employeeSet.add(emp4);
```

```
employeeSet.add(emp5); // Should not be added if equals to emp1
```

```
System.out.println("Set size: " + employeeSet.size());

System.out.println("All employees in set:");

for (Employee emp : employeeSet) {

    System.out.println(" " + emp);

}

System.out.println();


// Test comparing different objects

System.out.println("=== Additional Comparisons ===");

Employee emp6 = new Employee(102, "Jane Smith", "HR", 65000.0);

System.out.println("emp2 equals emp6? " + emp2.equals(emp6));

System.out.println("emp2 hash code: " + emp2.hashCode());

System.out.println("emp6 hash code: " + emp6.hashCode());

}

}
```

Key Features of this Implementation:

1. Private Variables

- `id`, `name`, `department`, `salary` - all private to enforce encapsulation

2. Public Getters/Setters

- Getter and setter methods for each private variable

- Allows controlled access to private data

3. Overridden equals() Method

- Compares all fields (id, name, department, salary)
- Uses `Objects.equals()` for null-safe object comparison
- Checks for reference equality and class type

4. Overridden hashCode() Method

- Uses `Objects.hash()` for consistent hash code generation
- Uses same fields as equals() method
- Ensures contract: equal objects have equal hash codes

5. Overridden toString() Method

- Provides meaningful string representation
- Shows all employee details in readable format

Sample Output:

text

```
=== Testing Getters ===
Employee 1 ID: 101
Employee 1 Name: John Doe
Employee 1 Department: IT
Employee 1 Salary: $75000.0

=== Testing Setters ===
Before update: Employee{id=101, name='John Doe', department='IT', salary=75000.0}
After update: Employee{id=101, name='John Doe', department='IT - Senior', salary=78000.0}

=== Testing toString() ===
Employee 1: Employee{id=101, name='John Doe', department='IT - Senior', salary=78000.0}
Employee 2: Employee{id=102, name='Jane Smith', department='HR', salary=65000.0}
Employee 3: Employee{id=103, name='Bob Johnson', department='Finance', salary=80000.0}
Employee 4: Employee{id=104, name='Alice Brown', department='Marketing', salary=70000.0}
```

```
=== Testing equals() ===
emp1 equals emp2? false
emp1 equals emp5? true
emp1 equals null? false
emp1 equals itself? true

=== Testing hashCode() ===
emp1 hash code: -123456789
emp2 hash code: 987654321
emp5 hash code: -123456789
emp1 and emp5 have same hash code? true

=== HashSet Behavior (using equals/hashCode) ===
Set size: 4
All employees in set:
  Employee{id=104, name='Alice Brown', department='Marketing', salary=70000.0}
  Employee{id=101, name='John Doe', department='IT - Senior', salary=78000.0}
  Employee{id=102, name='Jane Smith', department='HR', salary=65000.0}
  Employee{id=103, name='Bob Johnson', department='Finance', salary=80000.0}
```

This implementation follows Java best practices for encapsulation, object comparison, and collection usage.